JUNIOR ENGINEER

Your First Month as a DevOps Engineer

A Week-by-Week Plan



Week 1
Understand the Environment

Week 2
Learn How Code Becomes a Running App

Week 3
Troubleshoot Without Making Things Worse

Week 4
Start Contributing Like a Reliable Engineer

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Day One: What Should a Junior DevOps Engineer Actually Do?

1 Understand Your Environment

Learn your team's responsibilities, applications, environments, and deployment ownership.

- What services do you support?
- Which environments exist (dev, staging, production)?
- Who owns each application?

2 Build Your Network

Identify your manager, technical mentor, service owners, and escalation contacts.

- Know who to ask for help.
- Understand the escalation path.
- Get familiar with your team structure.

3 Learn the Communication Channels

Find out where engineers communicate, document changes, and report incidents.

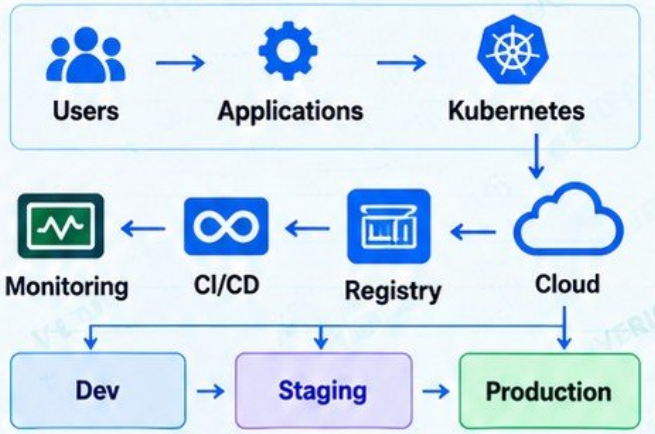
- Chat tools (Slack/Teams).
- Documentation (wiki, runbooks).
- Incident management (alerts, tickets).

4 Complete Your First Assignment

Create an onboarding checklist and record unanswered questions.

- List all access you need.
- Track what you still need to learn.
- Keep your questions for your mentor.

The DevOps Ecosystem



Junior Engineer Tip

Your first responsibility is understanding the system, not changing it.

Onboarding Checklist (Start Here)

- [ ] Access to GitHub, cloud, and clusters
- [ ] VPN and MFA setup
- [ ] Team communication channels
- [ ] Documentation links
- [ ] List of applications and environments
- [ ] Questions for your mentor

Learn the system.
Build your confidence.
You've got this!

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

Day Two: Getting Access Without Creating Security Risks

1 Understand the Access Tools

You will use different access methods to work safely in your DevOps role.

- **SSO** Single sign-on to access company tools.
- **MFA** Extra verification for your account.
- **VPN** Secure connection to internal systems.
- **SSH** Secure access to servers.
- **IAM** Identity and Access Management controls who can do what.
- **Role-based access** Gives you only the permissions you need.

2 Know Which Systems You Can Access

Not all systems allow changes. Access is based on your role and responsibility.

System	Typical Access	Purpose
GitHub	Read/Write (specific repos)	Code, configs, documentation
Cloud	Read/Write (limited)	Manage approved resources
Kubernetes	Read (start) Write (later)	Explore, then make approved changes
Production	Read-only (initially)	Observe and learn
Monitoring	Read-only	View logs, metrics, alerts
CI/CD	Read/Write (limited)	View and trigger approved jobs

3 Why Production Access Is Restricted

Production systems contain real customer data and critical services. Access is restricted to:

- Prevent accidental changes.
- Reduce the risk of security breaches.
- Ensure changes are reviewed and approved.
- Maintain accountability (audit logs).
- Follow company and regulatory requirements.

⚠ Administrator privileges are not given to everyone because one mistake can cause a major outage or data loss.

4 Practical Access Verification

After getting your accounts, verify they work correctly.

- **GitHub** `git config --global user.name` Checks your GitHub identity.
- **Cloud** `aws sts get-caller-identity` Confirms your cloud identity.
- **Kubernetes** `kubectl get nodes` Verifies cluster access.

5 Common Failure

Permission denied does not automatically mean you need administrator access.

```
$ kubectl get pods
Error from server (Forbidden):
pods is forbidden: User "dev-user"
cannot list resource "pods" ...
```

Common Reasons

- Your role does not have permission.
- You are using the wrong namespace.
- You need a different role (e.g. read-only).
- You are not authenticated correctly.
- The resource is restricted for your team.

What to Do Instead

- Check your current identity.
- Verify your role and namespace.
- Ask your team or mentor.
- Use the correct access method.
- Request the required permission through the proper process.

Junior Engineer Tip Access should be granted based on your responsibilities. Start with the minimum permissions you need. You can always request more access later through the proper process.

Day Three:
Meet the Infrastructure: Map What Runs Where

1 Understand the Big Picture

- Identify the application's components, environments, and hosting platform.
- Know which services run in your infrastructure.
- Understand how everything connects.
- Learn where your code, containers, and infrastructure live.

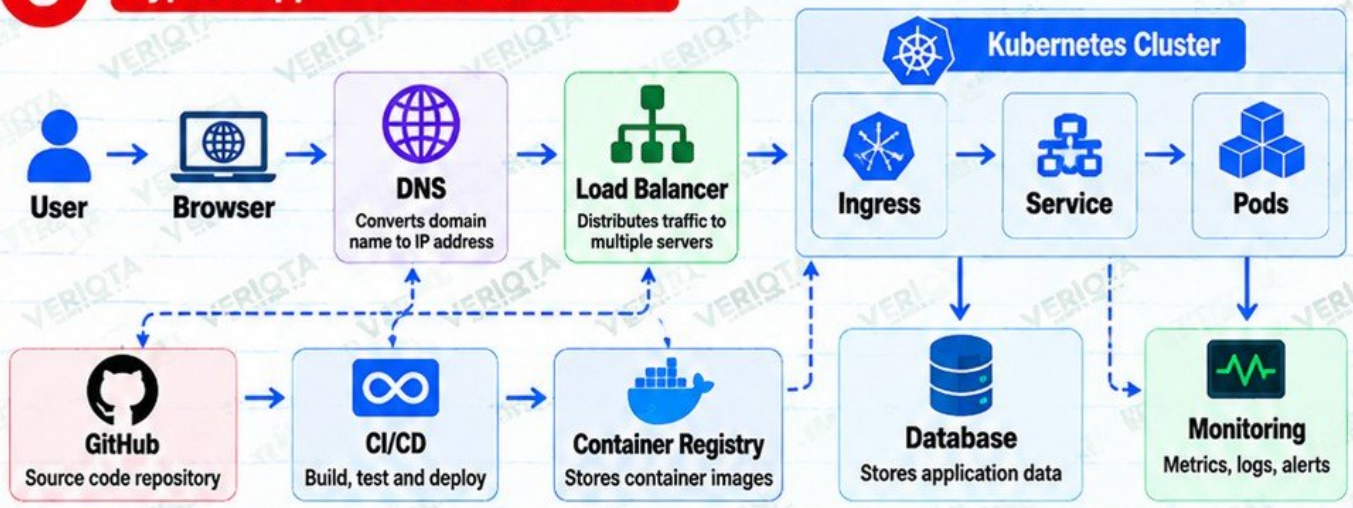
2 Environments You Will Work With

Development
- For active development
- Used by engineers
- Changes can be frequent
- May have sample data

Staging
- Looks like production
- Used for testing
- Verify changes before production
- Helps reduce risk

Production
- Live environment
- Real users
- Most stable
- Changes are controlled and approved

3 Typical Application Infrastructure



4 Key Components to Identify

- **Kubernetes Clusters** — Where containerized applications run.
- **Container Registry** — Stores your applications images.
- **Cloud Platform** — e.g. AWS, Azure or GCP.
- **Monitoring Tools** — e.g. Prometheus, Grafana, CloudWatch.
- **Infrastructure Repositories** — e.g. Terraform, Kubernates manifests, scripts.

5 Your First Infrastructure Map

Your Deliverable

- Draw a simple diagram of how the application works.
- Include main components.
- Show dev, staging and production environments.
- Add tools used (GitHub, CI/CD, registry, monitoring, etc).
- Confirm details with your mentor.

6 Junior Engineer Tip

Your first responsibility is understanding the system, not changing it.
A clear mental model helps you make better decisions and avoid mistakes.

Day Four: Your First Linux Investigation

1 Navigate a Server

Use these basic commands to move arounnt d and view files.

Command	What It Does	Example
`pwd`	Shows your current directory.	`pwd`
`ls`	Lists files and directories.	`ls -lah`
`cd`	Change directory.	`cd /var/log`
`cat`	View file content.	`cat app.log`

2 Inspect the System

Check users, processes, disk space and memory.

Command	What It Does	Example
`whoami`	Shows your current user.	`whoami`
`ps`	View running processes.	`ps aux`
`df -h`	Check disk space usage.	`df -h`
`free -h`	Check memory usage.	`free -h`

3 Observing vs Modifying

As a junior engineer, you will spend a lot of time observing the system. Avoid unnecessary changes, especially on production servers.

Observing (Safe)

- Viewing files and logs
- Checking system state
- Inspecting processes
- Checking disk and memory
- Reading configuration

Modifiying (Be Careful)

- Installing packages
- Editing configuration
- Restarting services
- Deleting files
- Changing permissions

4 Troubleshooting Scenario

A service is slow. What should you inspect first?

1. Check if the service is running. `ps aux | grep <service>`
2. Check resource usage (CPU, memory, disk).
```
top
df -h
free -h
```
3. Check application logs for errors. `cat /var/log/<service>/app.log`
4. Check recent changes or deployments. `cat /var/log/syslog | tail -n 50`
5. If needed, escalate with the evidence.

5 Practice: Collect Basic System Information

Run these commands on a non-production server and save the output to a file.

```
# Create a folder for your info
mkdir ~/linux-investigation
cd ~/linux-investigation

# Collect basic system information
whoami > user.txt
hostname > hostname.txt
uname -a > system.txt
df -h > disk.txt
free -h > memory.txt
ps aux --sort=-%cpu | head -n 10 > processes.txt
ls -lah /var/log > logs.txt
```

6 Example Output (Shortened)

whoami

```
devops
```

df -h

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   18G   30G  38% /
```

ps aux | head -n 3

```
USER         PID %CPU %MEM    COMMAND
root           1  0.0  0.1    systemd
devops      1254  0.5  1.2    python
devops      2387  0.1  0.3    nginx
```

free -h

```
               total        used        free
Mem:           7.6G        2.1G        5.5G
Swap:          2.0G          0B        2.0G
```

7 Common Failures

- Command not found → Tool not installed or wrong command.
- Permission denied → You may not have the right permissions.
- No such file or directory → Wrong file path.
- Disk space full → No space left on the server.
- High CPU or memory → Process using too many resources.

Junior Engineer Tip

Your first responsibility is understanding the system, not changing it. Gather evidence, ask questions, and make informed recommendations.

Your First Week Review: Can You Explain the System?

1 Trace the Path: Repository to Production

Understand how code moves from your repository to a running application in production.

GitHub
Repository (code)

→

CI/CD
Build, test, scan, package

→

Container Registry
Store images

→

Kubernetes
Deploy & run

→

Cloud
Infrastructure & networking

→

Application
Running in production

2 Who Approves Deployments and Rollbacks?

Learn the approval process and where rollback instructiong are kept.

Deployment Approvals

- Pull request review
- CI/CD pipeline checks
- Team lead /service owner
- Change approval (if required)

Rollback Instructions

- Runbook
- CI/CD pipeline
- Kubernetes manifests
- Infrastructure repo

3 Common Gaps to Watch For

Look for missing access, outdated docs, and unclear ownership.

Missing Access
You can't see the tools, clusters, or logs you need.

Outdated Documentation
Instructions don't match the current system.

Unclear Ownership
No one is clearly responsible for a service or application.

4 Your System-Understanding Report

Create a short report with verified facts and open questions.

Verified Facts

- What you have confirmed
- What works
- What you can access
- What you understand

Open Questions

- What is unclear
- What you need to learn
- Who to ask
- What you need access to

5 Week 1 Checkpoint

Explain the environment to your mentor without guesssing.

- ☑ Walk through the application flow.
- ☑ Explain the environments (dev, staging, prod).
- ☑ Name the key tools and services.
- ☑ Mention who approves changes.
- ☑ Share your questions and next steps.

Junior Engineer Tip | Your first responsibility is understanding the system, not changing it.

Day Six:
Your First Repository: Read Before You Commit

1 Explore a Real Repository

Get familiar with the structure and key files in an application or infrastructure repository.

```
sample-app/
├── .github/      CI/CD workflows
├── app/          Application code
├── config/       Configuration files
├── k8s/          Kubernetes manifests
├── docker/       Dockerfile and related files
├── scripts/      Helper scripts
├── README.md     Project overview
├── .gitignore    Files to ignore
└── LICENSE       License information
```

2 Understand Important Files and Directories

Each file or directory has a purpose. Read them to understand the project.

File / Directory	Purpose
README.md	Project overview, setup steps, usage instructions.
Application code	Source code for the application.
Dockerfile	Instructions to build the container image.
Workflows (.github/workflows/)	CI/CD pipeline definitions.
Configuration directories	Environment variables, config files, infrastructure settings.
Kubernetes manifests (k8s/, k8s/manifests/)	Deployment, Service, Ingress, ConfigMap, etc.

3 Useful Git Commands

These commands help you explore the repository safely.

Command	What It Does
`git status`	Shows the current branch and any changes.
`git log --oneline`	Shows recent commits in a short format.
`git branch`	Lists local branches.
`git branch -r`	Lists remote branches.
`git remote -v`	Shows remote repository URLs.
`git show <commit>`	View details of a specific commit.
`git diff`	See changes between files.

4 Why History and Contribution Instructions Matter

- The commit history shows what changed, when, and why.
- You can learn from past decisions and avoid repeating mistakes.
- Contribution instructions explain how to make changes correctly.
- They help you follow coding standards, testing requirements, and review processes.
- A good history makes troubleshooting and rollback easier when something goes wrong.

5 Practice: Find the Deployment Configuration

Locate the files that define how the application is deployed and explain their purpose.

1. Find the Dockerfile. What does it do?
2. Locate the Kubernetes manifests (e.g. k8s/).
3. Identify the CI/CD workflow file (e.g. .github/workflows/).
4. Explain how these files work together to deploy the application.
5. Write 3–5 key points about what you learned.

Junior Engineer Tip

- Always read the README.md first.
- Understand the project structure before making changes.
- Explore and ask questions if something is unclear.
- Do not commit changes until you fully understand the process.
- A little time spent reading can save you from bigger problems later.

Day Seven:
Your First Pull Request: Make a Safe Change

1 Understand the Key Concepts

These are the building blocks of a pull request.

Branch	A separate line of development for your changes.
Commit	A snapshot of your changes with a message.
Pull Request	A request to merge your changes into the main branch.
Code Review	Team members check your changes before merging.
Protected Branch	Limits who can push or merge directly (e.g. `main`).

2 Make a Small and Safe Change

Start with a simple change like updating documentation or a non-production configuration file.

```
# 1. Create a new branch
git checkout -b docs/update-readme

# 2. Make your changes (e.g. edit README.md)

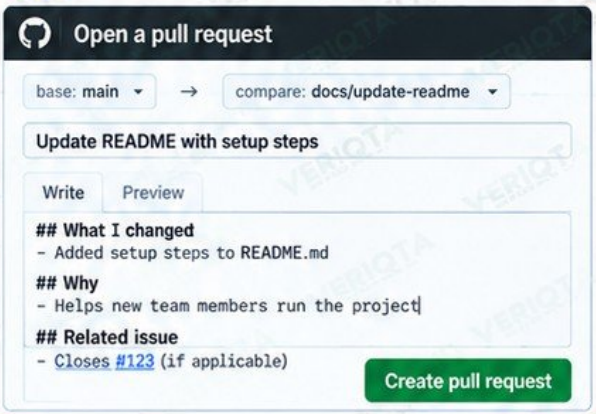
# 3. Stage and commit
git add README.md
git commit -m "Update README with setup steps"

# 4. Push your branch
git push -u origin docs/update-readme
```

3 Open a Pull Request

Go to your repository and create a pull request.

1. Select your branch.
2. Add a clear title and description.
3. Explain what you changed and why.
4. Link any related issue (if available).
5. Request review from the right team members.



4 Run Checks Before Requesting Review

Make sure your changes are valid and will not break anything.

- Check for code or configuration errors.
- Run linters or tests (if available).
- Follow the repository's contribution guidelines.
- Ensure you are not committing secrets.
- Confirm your changes are in the correct branch.

5 Respond to Review Feedback

Be open to feedback and make the required changes.

- Read the comments carefully.
- Ask questions if something is unclear.
- Make the requested changes and push new commits.
- Keep the discussion professional and positive.
- Once approved, your pull request can be merged.

6 Avoid Committing Secrets

Never include sensitive information in your commits.

- Passwords, API keys, tokens
- Database connection strings
- Cloud credentials
- Private certificates
- Any sensitive configuration files

7 Deliverable

- One pull request that is reviewed and approved.
- A clear description of what you changed and why.
- Verification evidence (e.g. tests passed, preview link, or screenshot).

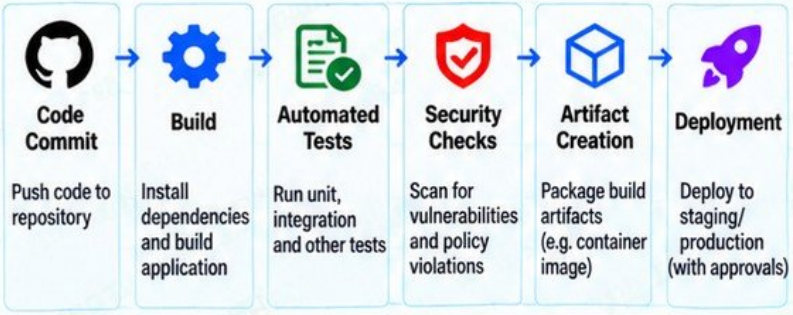
8 Junior Engineer Tip

Start small, follow the process, and focus on learning. A good pull request shows that you can work safely with the team and follow best practices.

Day Eight:
Follow a CI/CD Pipeline From Commit to Deployment

1 The End-to-End Flow

Trace how your code moves from a commit to a running application in production.



2 Key Components

These are the main parts of a CI/CD pipeline.

Component	What It Does
Pipeline Jobs	Individual tasks (build, test, scan, deploy).
Runners	Machines that execute the jobs (e.g. GitHub Actions runner).
Build Artifacts	Output from the build (e.g. JAR, Docker image, compiled files).
Security Checks	Scan for vulnerabilities, secrets and policy issues.
Deployment Approvals	Manual or automated approval before deploying to production.

3 Example Pipeline (GitHub Actions)

A simple pipeline with build, test, scan and deploy stages.

```yaml
name: CI-CD Pipeline
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Install dependencies
        run: npm install
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Run tests
        run: npm test
  deploy:
    needs: [build, test]
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to staging
        run: echo "Deploying..."
```

4 When a Job Fails

Learn how to locate failed jobs and read the logs.

1. Open the pipeline run.
2. Find the job that failed (red icon).
3. Click the job to view logs.
4. Read the error message carefully.
5. Identify the cause (e.g. test failure, missing package, configuration error).
6. Fix the issue and re-run the pipeline.

5 Failure Scenario: Tests Failed

The pipeline is red because a test failed.

```
> Run npm test
> app@1.0.0 test
> jest
 FAIL  src/login.test.js
  ✕ should return 200 for valid login (102 ms)

  Expected status code 200
  Received 401

    at Object.<anonymou>
    (src/login.test.js:24:12)
```

Why Deployment Should Stop

- Tests ensure the application works correctly.
- A failing test means there is a problem that could affect users.
- Deploying broken code can cause outages, security issues or data loss.
- Fix the issue, make the tests pass, and re-run the pipeline.

6 Practice: Identify the Failing Stage

Look at the pipeline and answer:

1. Which stage failed?
2. What is the error message?
3. What might be the cause?
4. What needs to be fixed?
5. Why should deployment not continue?

7 Key Takeaways

- Understand each stage in the pipeline.
- Know what runs in each job.
- Check logs carefully when something fails.
- Never ignore failing tests.
- Deployment should only happen when all required checks pass.

Junior Engineer Tip

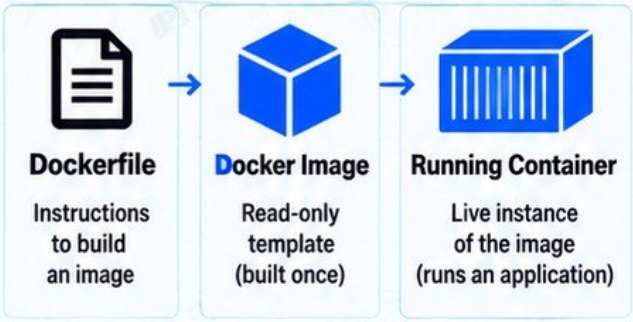
A red pipeline is a safety feature, not a failure. It prevents bad code from reaching your users. Learn to read the logs, fix the issue, and make the pipeline green again.

Day Nine:

Containers: Why It Works Locally but Fails Elsewhere

1 Docker Image vs Running Container

A Docker image is a read-only template.
A running container is a live instance of that image.



2 Inspect and Troubleshoot Containers

Use these commands to understand what is running and why it might be failing.

Command	What It Does
`docker ps -a`	List all containers (running and stopped).
`docker logs <container>`	View container logs.
`docker inspect <container>`	Show detailed information about a container.
`docker exec -it <container> sh`	Open a shell inside a running container.
`docker rm <container>`	Remove a stopped container.
`docker images`	List available images.

3 Environment Variables

Containers often use environment variables for configuration (e.g. database URL, API keys).

```
# View environment variables for a container
docker inspect <container> | grep -A 10 "Env"

# Example when running a container
docker run -e APP_ENV=dev -e PORT=8080 myapp:latest
```

4 Ports and Persistent Storage

Containers need correct port configuration and storage.

Exposed Ports

Map container ports to host ports so the application is accessible.

```
docker run -p 8080:80 myapp:latest
```

This maps port 80 in the container to port 8080 on your host.

Persistent Storage

Use volumes to keep data even if the container stops or is removed.

```
docker run -v app-data:/data myapp:latest
```

This mounts a named volume called app-data to /data in the container.

5 Troubleshoot a Container that Exits Immediately

A container that exits quickly usually has an error.

1. Check the container logs: `docker logs <container>`
2. Check the container's command and entrypoint.
3. Verify required environment variables are set.
4. Confirm the application dependencies are available.
5. Run the container interactively to debug: `docker run -it <image> sh`
6. Look for configuration or file permission issues.

6 Common Reasons It Works Locally but Fails Elsewhere

- ✗ Missing environment variables (e.g. database URL, API keys).
- ✗ Different configuration files or values.
- ✗ Port conflicts or incorrect exposed ports.
- ✗ Missing persistent storage or incorrect volume paths.
- ✗ Different application dependencies or versions.
- ✗ Resource limits (CPU or memory) in the target environment.
- ✗ Network or DNS issues when connecting to external services.

7 Production Awareness

Never assume a running container means a healthy application.

- Always check logs, health endpoints, and metrics.
- A container can be running but the application inside can still be failing.
- Use proper monitoring and alerts in production.

Day Ten:
Your First Kubernetes Deployment Investigation

1 Key Kubernetes Components

These are the main resources you will work with.



Pod

Runs one or more containers.



Deployment

Manages Pods and keeps the desired number running.



Service

Provides a stable network endpoint to access Pods.



Namespace

Logically separates resources in the cluster.

2 Useful kubectl Commands

Use these commands to inspect and understand workloads.

Command	What It Does
`kubectl get pods`	List all Pods in the current namespace.
`kubectl get pods -n <ns>`	List Pods in a specific namespace.
`kubectl describe pod <pod>`	Show detailed information about a Pod (events, status, errors).
`kubectl logs <pod>`	View application logs from a Pod.
`kubectl get deployments`	List all Deployments.
`kubectl get services`	List all Services.
`kubectl get all -n <ns>`	List all resources in a namespace.

3 How a Deployment Creates Pods

A Deployment manages ReplicaSets, which create and maintain Pods.



Deployment (Desired state: 3 replicas) → **ReplicaSet** (Manages Pods) → **Pods** (3 running)

4 Investigate ImagePullBackOff

This usually means the container image cannot be pulled (e.g. wrong image name, private registry, or missing credentials).

```
$ kubectl get pods
NAME                     READY   STATUS             RESTARTS   AGE
web-app-5d7f8c9b6-abc12  0/1     ImagePullBackOff   0          2m
```

Common Causes:

- Incorrect image name or tag.
- Image does not exist in the registry.
- Missing or incorrect registry credentials.
- No network access to the container registry.

5 Investigate CrashLoopBackOff

This means the container started but keeps crashing.

```
$ kubectl get pods
NAME                     READY   STATUS             RESTARTS   AGE
api-6f9d7b8c9-xyz34      0/1     CrashLoopBackOff   5          3m
```

Common Causes:

- Application error (e.g. missing environment variable).
- Incorrect configuration or command.
- Dependency not available (e.g. database, API).
- Insufficient resources (CPU or memory).
- Check logs with: `kubectl logs <pod>`

6 Follow the Deployment Process

View how a Deployment creates and manages Pods.

1. Check the Deployment: `kubectl get deployment <name>`
2. Describe the Deployment: `kubectl describe deployment <name>`
3. Check the ReplicaSet: `kubectl get rs`
4. Check the Pods: `kubectl get pods -l app=<name>`
5. View events if there are issues: `kubectl describe pod <pod>`

7 Week 2 Checkpoint



Explain to your mentor how a code change in the repository becomes a running workload in Kubernetes.

1. From commit to container image.
2. From image to Deployment.
3. From Deployment to running Pods.
4. How you would investigate if the Pods are not running.

Day Eleven:
Your First Alert: What Do You Check First?

1 Key Concepts

These tools help you understand what is happening in your system.

Metrics	Logs	Traces	Alerts	Dashboards
Quantitative data (CPU, memory, request rate, latency).	Detailed records of events and errors.	Show the path of a request across services.	Notify you when a metric breaches a threshold.	Visualize metrics, logs, and system health.

2 Symptom vs Root Cause

A symptom is what you see. The root cause is why it happened.

Symptom (What You See)	Root Cause (Why It Happened)
High CPU usage	Memory leak
Increased error rate	Large spike in traffic
Slow response time	Dependency failure
Service not reachable	Configuration change
Container restarting	Insufficient resources
	Bug in code

3 Common Alerts to Investigate

These are common alerts you may see in production.

High CPU Usage
- Check running processes.
- Look for traffic spikes.
- Find any recent changes.

High Memory Usage
- Check memory usage over time.
- Look for memory leaks.
- Check if the application is restarting.

High Error Rate
- Check application logs.
- Look for recent deployments.
- Identify failing endpoints.

High Response Latency
- Check dependency services.
- Look for database or network issues.
- Compare with traffic levels.

4 Use Dashboards to Investigate

Look at metrics, logs, and traces together to get the full picture.



5 Steps to Investigate an Alert

1. Read the alert details (service, metric, threshold).
2. Check the dashboard for the time period.
3. Look at relevant logs for errors.
4. Check recent deployments or configuration changes.
5. Investigate dependencies (database, external APIs).
6. Identify the most likely cause.
7. Decide if it requires escalation.

6 When to Escalate

Escalate when:

- The service is fully down or critical features are affected.
- You cannot identify the cause within a reasonable time.
- There is a security or data issue.
- You need additional permissions or expertise.
- The impact is high (many users or customers affected).

7 Practice: Document the Alert

Create a short incident report with the following information.

- Alert name and time.
- Affected service.
- What you observed (metrics, logs, errors).
- Most likely cause.
- Action taken and next steps (escalate or resolve).

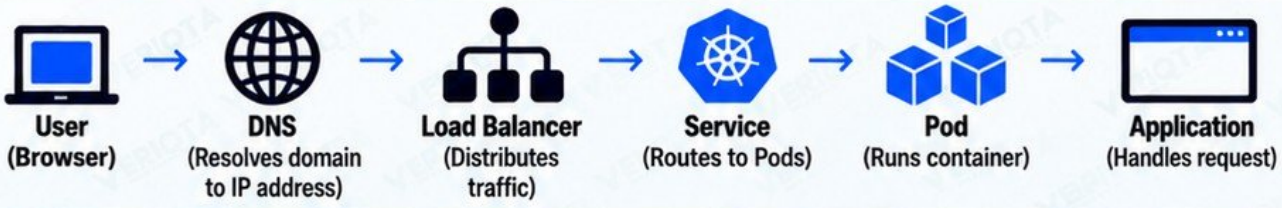
Junior Engineer Tip

- Do not panic when you see an alert.
- Use evidence, not guesses.
- Check metrics, logs, and traces.
- Understand the difference between symptoms and root cause.
- Document what you find.

Day Twelve:
The Application Is Down: Trace the Request

1 Follow the Request Path

A user's request passes through multiple components before reaching your application.



2 Common Failure Types

These are common issues when an application is down.

Failure Type	What It Means	Example Error
DNS Failure	Domain cannot be resolved to an IP address.	`dig: no such host`
Connection Refused	Service is not accepting connections.	`curl: (7) Failed to connect`
Timeout	Request takes too long and fails.	`curl: (28) Operation timed out`
HTTP 5xx	Server responded with an error.	`HTTP/1.1 502 Bad Gateway`

3 Useful Commands

Use these commands to investigate each part of the path.

Check DNS	`dig example.com`
Test application endpoint	`curl -v https://example.com`
Check Kubernetes Service	`kubectl get service`
Check Pods	`kubectl get pods`
Describe a Pod	`kubectl describe pod <pod-name>`
View Pod logs	`kubectl logs <pod-name>`
Check Endpoints	`kubectl get endpoints`
Check Ingress	`kubectl get ingress`

4 Investigate Step by Step

Follow this order to identify where the problem is.

1. **Check DNS:** Does the domain resolve to the correct IP?
2. **Check Load Balancer:** Is it reachable and healthy?
3. **Check Service:** Is it routing to the correct Pods?
4. **Check Pods:** Are the Pods running and healthy?
5. **Check Application:** Is the application returning a successful response?

5 Example Troubleshooting Flow

A real example when the application is down.

```
$ dig example.com
example.com.      300     IN      A       203.0.113.10
$ curl -v https://example.com
> GET / HTTP/1.1
< HTTP/1.1 502 Bad Gateway
$ kubectl get pods
NAME                          READY   STATUS             RESTARTS   AGE
web-app-7d9f8c6c9-abc12       0/1     CrashLoopBackOff   5          3m
```

6 Key Questions to Ask

Answer these questions based on your investigation.

- Does the domain resolve to the correct IP address?
- Can you reach the load balancer?
- Are the Kubernetes Services and Endpoints correct?
- Are the Pods running and healthy?
- Are there errors in the application logs?
- Is the application returning the expected response?

7 Troubleshooting Tip

- A reachable server does not prove the application works.
- Always check the full request path, not just one component.
- Use evidence (logs, metrics, commands) before proposing a fix.
- Identify the failing component first, then investigate the root cause.

8 Production Awareness

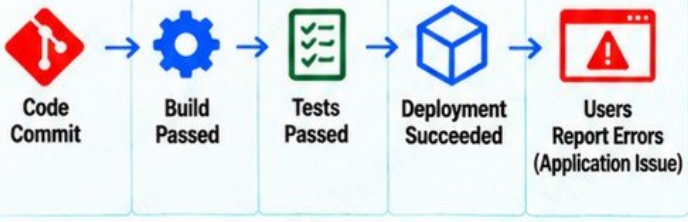
- Check timeouts, retries, and health checks.
- 5xx errors may indicate a dependency issue.
- Multiple issues can occur at the same time.
- Understand the impact on users before making changes.
- Escalate if you are unsure or the impact is high.

Day Thirteen:

The Deployment Succeeded, but Users Are Reporting Errors

1 Why This Happens

A successful CI/CD pipeline only means the deployment was completed. It does not guarantee the application is healthy or working correctly.



2 What to Inspect First

Check these areas to find the cause of the errors.

Readiness Probes	Ensure the application is actually ready to serve traffic.
Application Logs	Look for errors, stack traces and recent changes.
Error Metrics	Check error rates, response times and failed requests.
Recent Changes	Review what changed in this deployment (code, configuration, dependency, environment).

3 Useful Commands

Use these commands to investigate the application.

```
# Check deployment status
kubectl get deployments

# Check pod status and readiness
kubectl get pods

# Describe a pod (look at events, probes, errors)
kubectl describe pod <pod-name>

# View application logs
kubectl logs <pod-name> --tail=100

# Check recent changes in the deployment
kubectl rollout history deployment <name>

# Check previous version details
kubectl rollout history deployment <name> --revision=<n>
```

4 Compare With the Previous Healthy Version

Find out what is different between the working version and the current version.

What to Compare	Examples
Code Changes	New features, bug fixes, configuration changes.
Environment Variables	Missing or incorrect values.
Dependencies	New library or version update.
Configuration Files	Different settings or limits.
Infrastructure	Resource limits, node changes.
Application Behavior	Error rates, response time, failed endpoints.

5 When to Stop or Roll Back

Consider stopping the rollout or rolling back when:

- Error rate has significantly increased.
- Core user functionality is broken.
- The issue is caused by the recent deployment.
- There is risk to users, data, or business operations.
- You cannot quickly identify or fix the issue.
- Your team's runbook or policy allows rollback.

6 Deployment Verification Checklist

Use this checklist after every deployment.

- Pods are running and ready (readiness probes passing).
- Application logs show no new errors.
- Key endpoints return the expected response.
- Error rate and latency are within normal range.
- Core user functionality works as expected.
- Compare with previous healthy version.
- Monitor for a period of time after deployment.
- Confirm with the team before closing the change.

7 Key Takeaways

- A successful deployment is not the same as a healthy application.
- Always verify the application after deployment.
- Use logs, metrics and probes to find the real cause.
- Compare with the previous version to identify changes.
- Know when to stop, roll back or escalate.

Production Awareness

- Users can experience errors even when the pipeline is green.
- Monitor the application after every deployment.
- Have a rollback plan and follow your team's process.
- Communicate clearly with your team and stakeholders.

Day Fourteen:

Your First Incident: What Happens in the First 10 Minutes?

1 What Is an Incident?

An incident is an unplanned disruption or degradation in a service that impacts users, business operations, or system functionality.

Critical (Severity 1)	High (Severity 2)	Medium (Severity 3)	Low (Severity 4)
• Major outage • Many users affected • Immediate response • Executive visibility	• Significant impact • Core features affected • Quick response • Management notified	• Limited impact • Non-critical features • Normal response • Team handles	• Minimal impact • Cosmetic or minor issue • Routine response • Track and resolve

2 Key Roles During an Incident

Different people have different responsibilities.

Role	What They Do
Incident Commander (IC)	• Owns the incident response • Coordinates the team • Makes key decisions • Communicates with stakeholders
Investigator	• Finds the root cause • Gathers evidence (logs, metrics, etc.) • Proposes possible fixes • Works with the IC
Communicator	• Shares updates with stakeholders • Writes clear and accurate messages • Keeps everyone informed • Avoids speculation

3 Your First 10 Minutes: Step by Step

Follow these steps when an incident is declared.

1. **Understand the alert**
   Read the alert details and check the impact.
2. **Confirm the issue**
   Reproduce the issue if possible.
3. **Check recent changes**
   Look at deployments, configuration changes, or infrastructure updates.
4. **Gather initial evidence**
   Check dashboards, logs, and error messages.
5. **Communicate**
   Share what you know with the team (confirmed facts only).
6. **Follow instructions**
   Wait for guidance from the incident commander.

4 Record Important Information

Keep a timeline of everything you observe and do.

Time (UTC)	What to Record	Example
00:00	Alert received	High error rate on /api
00:02	Initial observation	Users cannot log in
00:04	Impact assessment	Affects all regions
00:06	Recent changes	New deployment 10 mins ago
00:08	Checks performed	Checked logs and metrics
00:10	Current status	Issue confirmed, investigating

5 What Not to Do as a Junior Engineer

Avoid these common mistakes during an incident.

- ✗ Do not make unapproved changes in production.
- ✗ Do not assume the cause without evidence.
- ✗ Do not restart services without approval.
- ✗ Do not ignore communication from the incident commander.
- ✗ Do not work on the issue alone.
- ✗ Do not share unverified information with users or stakeholders.

6 Example Incident Update

A concise and clear update with confirmed facts.

Incident:	Login service errors
Severity:	High (Severity 2)
Time:	2026-10-15 14:05 UTC
Impact:	Users cannot log in (web and mobile)
What We Know:	• Error rate increased to 80% • Started 10 minutes after deployment • Affects all regions
What We Are Doing:	• Investigating application logs • Checking recent changes • Working with the incident commander
Next Update:	In 10 minutes or sooner if there is a change

7 Key Takeaways

- Understand the severity and impact of the incident.
- Know your role and follow the incident commander's instructions.
- Record a clear timeline with facts.
- Use evidence from dashboards, logs, and metrics.
- Do not make production changes without approval.
- Communicate clearly and avoid speculation.

8 Remember

- The first 10 minutes set the direction for the incident.
- Stay calm and focus on verified facts.
- Junior engineers add value by gathering information, not by making unapproved changes.
- Clear communication helps the team resolve the incident faster.

Day Fifteen:

Security and Recovery: Protect the System While Fixing It

1 Common Security Risks

Recognize these common security risks in deployments and configurations.

Exposed Credentials
Hardcoded passwords, API keys or tokens in code or configuration.

Excessive Permissions
Using admin or overly broad permissions instead of least privilege.

Unsafe Configuration
Public access, open ports, weak security settings.

Untrusted Images
Using unverified or outdated container images from public registries.

Missing Audit Trails
No logging or monitoring of important actions.

2 Least Privilege and Secrets Management

Follow these practices to keep the system secure.

Practice	What It Means
Least Privilege	Give only the minimum permissions needed to do the job.
Secrets Management	Store secrets in a secure tool (e.g. Kubernetes Secrets, cloud secret manager).
Rotate Credentials	Change keys and passwords regularly.
Access Control	Use roles and policies to restrict access.
Audit Trails	Keep records of who did what and when.

3 Example of an Insecure Configuration

Identify the security risks in this sample deployment.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 1
  template:
    spec:
      containers:
      - name: app
        image: myapp:latest
        env:
        - name: DB_PASSWORD
          value: "mypassword123"
        ports:
        - containerPort: 8080
```

Security Risks:

- Hardcoded password in the configuration.
- Using latest image tag instead of a specific version.
- No resource limits.
- No security context.
- Pod accessible on a public port without restrictions.

4 Backups and Recovery

Backups are only useful if you can restore from them.

Regular Backups
Keep backups of important data and configurations.

Test Restores
Regularly test that you can actually restore the backups.

Verify Rollbacks
After a rollback, confirm that the application is working correctly.

Keep Records
Document backup and recovery procedures.

5 Week 3 Checkpoint

Explain how to investigate a failure without increasing its impact.

- Collect information using read-only commands.
- Avoid making unapproved changes in production.
- Check logs, metrics, and configuration safely.
- Identify the possible cause.
- Propose a fix and follow the proper approval process.
- Consider the security impact of any action.
- Communicate findings clearly to the team.

6 Security-Aware Troubleshooting Tips

1. Use read-only access when investigating.
2. Do not expose additional ports or services.
3. Avoid sharing sensitive information in chat tools.
4. Check for unusual activity in logs.
5. Validate the source of container images.
6. Ensure that any fix does not reduce security.

7 Key Takeaways

- Security and reliability go together.
- Protect the system while fixing the issue.
- Use least privilege and secure secrets management.
- Always test backups and verify rollbacks.
- Investigate failures carefully and avoid increasing the impact.

8 Practice Task

- [ ] Review the sample deployment configuration.
- [ ] Identify at least 5 security risks.
- [ ] Suggest specific improvements.
- [ ] Explain how you would investigate a failure safely.

Day Sixteen:

Infrastructure as Code: Understand Before You Apply

1 Why Use Terraform?

Teams use Terraform to manage cloud infrastructure as code so it is reproducible, consistent and version controlled.

Consistency	Collaboration	Version Control	Repeatability	Reduced Human Error
Same infrastructure every time.	Multiple engineers can work together.	Track changes in Git.	Recreate environments quickly.	Less manual configuration.

2 Key Concepts

Understand the main parts of a Terraform project.

Configuration Files	Written in HCL (e.g. main.tf) to define resources.
Providers	Plugin that lets Terraform interact with a cloud (e.g. AWS, Azure).
Resources	The actual infrastructure components (e.g. VPC, EC2, S3, IAM).
State	Keeps track of what Terraform has created. Stored remotely or locally.
Plan	Shows what changes will be made before applying.

3 Common Terraform Commands

These commands help you work safely and understand what will happen.

Command	What It Does
`terraform fmt`	Formats your configuration files to a standard style.
`terraform validate`	Checks for syntax errors and configuration issues.
`terraform plan`	Shows the changes Terraform will make (preview).
`terraform apply`	Creates, updates or destroys real infrastructure (make changes).

4 Sample Configuration File (main.tf)

This file defines an example AWS S3 bucket.

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "example" {
  bucket = "veriqta-demo-bucket"
  tags = {
    Name = "veriqta-demo"
    Environment = "dev"
  }
}
```

5 Sample Terraform Plan Output

A plan shows what Terraform will do without making changes.

```
Terraform will perform the following actions:
  # aws_s3_bucket.example will be created
  + resource "aws_s3_bucket" "example"  {
      + bucket    = "veriqta-demo-bucket"
      + tags      = {
          + Environment = "dev"
          + Name        = "veriqta-demo"
        }
    }
Plan: 1 to add, 0 to change, 0 to destroy.
```

6 What to Look for in a Plan

Inspect the plan output and identify the proposed changes.

Resources to be created	Look for lines starting with +.
Resources to be changed	Look for lines starting with ~.
Resources to be destroyed	Look for lines starting with -.
Details of each change	Review attributes, tags, and settings.
Unexpected changes	Check for resources you did not intend to modify.

7 Why terraform apply Can Be Risky

- It creates, updates or deletes real infrastructure.
- It can incur unexpected costs.
- It can affect live environments.
- Mistakes in configuration can cause serious impact.
- Always review the plan and get approval before applying.

8 Practice Task

- [ ] Review the sample configuration file.
- [ ] Run **terraform** fmt, terraform validate and **terraform plan** (on a safe environment or example files).
- [ ] Inspect the plan output and identify what will be created, changed or destroyed.
- [ ] List any potential risks you see in the proposed changes.

Day Seventeen: Your First Useful Automation

VERIQTA

1 Find a Repetitive, Low-Risk Task

Start with a simple task that you do often and that is safe to automate.

- Check disk usage
- Check memory usage
- Check running processes
- Collect system info
- Schedule regular reports

2 Write a Bash Script

This script collects disk, memory, and process information.

```bash
#!/bin/bash
# system-report.sh - Collect disk, memory and process info
set -e

# Output file with timestamp
TIMESTAMP=$(date +"%Y-%m-%d_%H-%M-%S")
OUTPUT="system_report_$TIMESTAMP.txt"

# Collect system information
{
    echo "==== System Report ===="
    echo "Date: $(date)"
    echo
    echo "--- Disk Usage ---"
    df -h
    echo
    echo "--- Memory Usage ---"
    free -h
    echo
    echo "--- Running Processes (Top 10 by CPU) ---"
    ps -eo pid,ppid,cmd,%cpu,%mem --sort=-%cpu | head -n 10
} > "$OUTPUT"

echo "Report saved to $OUTPUT"
exit 0
```

3 Make It User-Friendly and Safe

Add clear output, error handling and meaningful exit codes.

Feature	What to Do
Readable output	Use clear headings and timestamps.
Error handling	Check if commands exist and handle failures.
Meaningful exit codes	exit 0 = success exit 1 = general error exit 2 = missing command or permission
Input validation	Check required inputs (e.g. file path).
Safe commands	Use read-only commands for this task.

4 Test in a Non-Production Environment

Run the script on a test server or your local machine first.

- Run the script and check the output file.
- Confirm all sections show correct information.
- Test error scenarios (e.g. missing command).
- Check the exit code using echo $?.
- Make sure it does not affect the system.

5 Example Output

This is what the script output file looks like.

```
====  System Report ====
Date: 2026-10-15 14:30:00

--- Disk Usage ---
Filesystem      Size  Used  Avail Use%  Mounted on
/dev/sda1        50G   20G    28G   42%  /

--- Memory Usage ---
               total        used        free      shared  buff/cache   available
Mem:           7.7G        3.2G        1.8G        0.4G        2.7G        4.1G
Swap:          2.0G          0B        2.0G

--- Running Processes (Top 10 by CPU) ---
PID    PPID   CMD                         %CPU   %MEM
1234      1   /usr/bin/python             12.5    2.1
5678      1   /usr/bin/java                8.3    1.8
```

6 Document the Script

Create a simple README so another engineer can safely run it.

Include:

- Purpose of the script.
- Requirements (e.g. Linux, Bash).
- How to run it.
- Example output.
- Exit codes explanation.
- Any limitations or notes.

7 Key Takeaways

- Start with a small, useful and low-risk task.
- Write clear and readable scripts.
- Always handle errors and use meaningful exit codes.
- Test in a non-production environment.
- Document your script so others can use it safely.

8 Practice Task

1. Write the script shown above.
2. Run it and check the output.
3. Test an error scenario (e.g. remove a command).
4. Check the exit code.
5. Create a README with usage instructions.

VERIQTA

Day Eighteen:

Make a Change That Is Safe to Deploy and Easy to Reverse

1 Key Concepts to Understand

Understand the main areas before making a change.

Change Scope	Peer Review	Deployment Windows	Risk Assessment
Keep the change small and focused.	Get another engineer to review the change.	Make changes at the right time (usually low traffic).	Identify what can go wrong and how to reduce the risk.

2 Pre-Deployment Checklist

Complete these checks before deploying.

- [x] Change is reviewed and approved.
- [x] Testing is completed in a non-production environment (e.g. staging).
- [x] Rollback plan is prepared and tested.
- [x] Required monitoring and alerts are in place.
- [x] Dependencies and required permissions are confirmed.
- [x] Deployment window is agreed.
- [x] Success criteria are defined and measurable.
- [x] Team members are informed.

3 Rollback vs Fixing Forward

Understand when to roll back and when to fix forward.

Rollback

- Restore to a previous working version.
- Use when the change causes serious issues.
- Faster way to recover service.
- Requires a tested and available rollback plan.

Fixing Forward

- Make a new change to fix the issue.
- Use when rollback is not possible or not ideal.
- May be needed for database changes or complex updates.
- Still requires testing, review and monitoring.

4 Deployment Flow (Example)

A safe and controlled deployment process.

Make a small change → Test in staging environment → Verify it meets the success criteria → Deploy to production (in agreed window) → Monitor and verify after deployment

5 Record and Verify the Results

After the deployment, check that everything is working.

What to Check	Example
Application is running	kubectl get pods
No increase in error rate	Check logs and monitoring
Performance is normal	Compare latency and response time
Key user functionality works	Test main features
No unexpected side effects	Check related services
Record the results	Document what was changed and the outcome

6 Practice Task

Prepare a change plan for a staging deployment.

1. Define a small and safe change to make.
2. Write the purpose of the change.
3. Identify the steps to test it in staging.
4. Define the success criteria.
5. Write a rollback plan.
6. List what you will monitor after the change.
7. Share the plan for review.

7 Key Takeaways

- Keep changes small, well-tested and easy to reverse.
- Always use peer review and follow deployment procedures.
- Prepare a rollback plan before deploying.
- Verify the application and monitor after the change.
- Document the change and its results.

8 Success Criteria Examples

- Application responds with 200 OK.
- No increase in error rate.
- Response time stays within normal range.
- Key user journeys work as expected.
- No new critical alerts.

Day Nineteen:

Your First Runbook: Turn Investigation Into Team Knowledge

VERIQTA

1 What Is a Runbook?

A runbook is a step-by-step guide that helps engineers investigate and resolve common issues.

Consistent Response
Everyone follows the same steps.

Faster Resolution
Reduces time during incidents.

Less Risk
Prevents mistakes and risky actions.

Knowledge Sharing
Turns individual knowledge into team knowledge.

2 What Makes a Runbook Useful?

A good runbook is clear, accurate and safe to follow.

- Clear description of the issue and symptoms.
- Prerequisites (tools, access, required permissions).
- Safe diagnostic commands with expected results.
- Step-by-step troubleshooting steps.
- Verification steps to confirm the issue is resolved.
- Recovery guidance if the issue cannot be fixed.
- Escalation criteria (when to involve others).
- Warnings about risky actions.
- Reviewed by another engineer for accuracy.

3 Example Runbook Structure

Use this structure for your runbook.

1	Title	Clear and specific name for the issue.
2	Overview	Short description and impact.
3	Symptoms	How the issue appears.
4	Prerequisites	Tools, access and permissions needed.
5	Diagnostic Steps	Commands to run with expected results.
6	Verification	Steps to confirm the issue is resolved.
7	Recovery	How to restore service if needed.
8	Escalation	When to involve the team or another team.
9	Warnings	Actions to avoid and potential risks.

4 Sample Diagnostic Commands (Kubernetes)

These commands help investigate a common application issue.

```
# Check pod status
kubectl get pods -n <namespace>
# Describe a pod
kubectl describe pod <pod-name> -n <namespace>
# View logs
kubectl logs <pod-name> -n <namespace> --tail=100
# Check recent events
kubectl get events -n <namespace> --sort-by=.lastTimestamp
# Check deployment and replica status
kubectl get deploy -n <namespace>
# Check service and endpoints
kubectl get svc,endpoints -n <namespace>
```

5 Expected Results and Next Steps

Compare the output with the expected result.

Check	Expected Result	If Not as Expected
Pod status	Running and Ready	Investigate events and logs.
Pod logs	No repeated errors	Check application and configuration.
Events	No critical errors	Look for image, resource or crash errors.
Service	Has endpoints	Check pod readiness and selectors.
Application	Responds successfully	Proceed to verification.

6 Verification and Recovery

Confirm the issue is resolved and know how to recover.

Verification Steps

- Pods are running and ready.
- Logs show normal behavior.
- Service has healthy endpoints.
- Application responds as expected.
- Monitoring shows normal metrics.

Recovery Guidance

- Restart the deployment.
- Roll back to a previous version.
- Revert a recent configuration change.
- Restore from a backup if needed.
- Follow escalation process if unresolved.

7 Deliverable

Create a runbook for diagnosing an unhealthy Kubernetes application.

- Include all sections listed above.
- Use clear and safe commands.
- Add expected results and troubleshooting steps.
- Include verification and recovery guidance.
- Have another engineer review it for accuracy.

8 Key Takeaways

- A runbook helps the team respond consistently.
- Include symptoms, commands, expected results and next steps.
- Always verify the fix and provide recovery options.
- Warn about risky actions.
- Let another engineer review your runbook.
- Turn your investigation into team knowledge.

Day Twenty:

VERIQTA

Day 30: Prove What You Can Do and Plan Your Next 60 Days

1 Present Your Completed Work

Prepare and present the key items you have built.

Infrastructure Map	Pull Request	Troubleshooting Notes	Automation Script	Runbook
Show the architecture and key components.	Link to your PR with a clear description of the change.	Show how you investigated and solved an issue.	Share your script and explain what it does.	Present your runbook for a common issue.

2 Demonstrate Investigation Skills

Show how you investigate a failing workload step by step.

Identify the issue → Gather evidence (logs, metrics, events) → Find the possible cause → Confirm with testing → Explain the resolution

Key Points to Cover

1. What you checked and why.
2. The commands or tools you used.
3. What you found in the logs, metrics or configuration.
4. How you confirmed the root cause.
5. How you fixed the issue and verified it was resolved.

3 Independent vs Supervised Tasks

Identify what you can do on your own and what still needs supervision.

Tasks You Can Perform Independently	Tasks Requiring Supervision
Use common Linux and Kubernetes commands for investigation.	Making changes to production environments.
Read logs, metrics and events to find issues.	Managing cloud infrastructure changes.
Run approved automation scripts.	Modifying security settings or access permissions.
Make small, safe changes in non-production environments.	Handling complex incidents.
Create and update documentation (runbooks).	Performing destructive actions (e.g. delete resources).

4 Build Your 30/60/90-Day Plan

Set clear and measurable engineering goals.

First 30 Days (Foundation)	Next 60 Days (Growth)	Next 90 Days (Readiness)
Strengthen Linux and Kubernetes skills.	Handle more complex incidents with support.	Work more independently on day-to-day tasks.
Improve troubleshooting speed and accuracy.	Work on cloud and infrastructure tasks.	Lead small projects or improvements.
Write more automation scripts.	Improve CI/CD and deployment skills.	Improve system design understandings.
Create and refine runbooks.	Take ownership of small features or changes.	Contribute to reliability, security and cost optimizations.
Contribute to team documentation.	Build monitoring and alert improvements.	Be ready for a mid-level engineering role.

5 Final Checkpoint

Be ready to explain the following clearly and confidently.

1. **One complete application delivery path**
   From code to running in Kubernetes, including CI/CD, deployment and verification.
2. **One failure investigation**
   A real or practice example showing how you found the root cause and resolved it.
3. **One safe change process**
   How you planned, tested and deployed a change with minimal risk, including rollback plan.

6 Deliverables Checklist

Make sure you have everything ready to share.

- Infrastructure map (diagram and explanation).
- Pull request (link and summary).
- Troubleshooting notes (commands, findings, resolution).
- Automation script (with documentation).
- Runbook (reviewed by another engineer).
- 30/60/90-day development plan (with measurable goals).
- Short presentation or written summary of what you learned.

7 Key Takeaways

- You have built practical skills and real work examples.
- Documentation and runbooks make you more effective.
- Investigation skills are more important than guessing.
- A clear plan helps you continue to grow.
- You are now ready to take on more responsibility step by step.

8 What's Next?

- Keep practicing and improving your skills.
- Take on slightly bigger and more complex tasks.
- Contribute to team projects and reliability improvements.
- Continue building your portfolio on GitHub.
- Review your 30/60/90-day plan regularly and update it.